

String Handling In java :

string literal in Java is basically a sequence of characters. These characters can be anything like alphabets, numbers or symbols which are enclosed with double quotes. So we can say String is a collection of alpha-numeric character.

How we can create String in Java :-

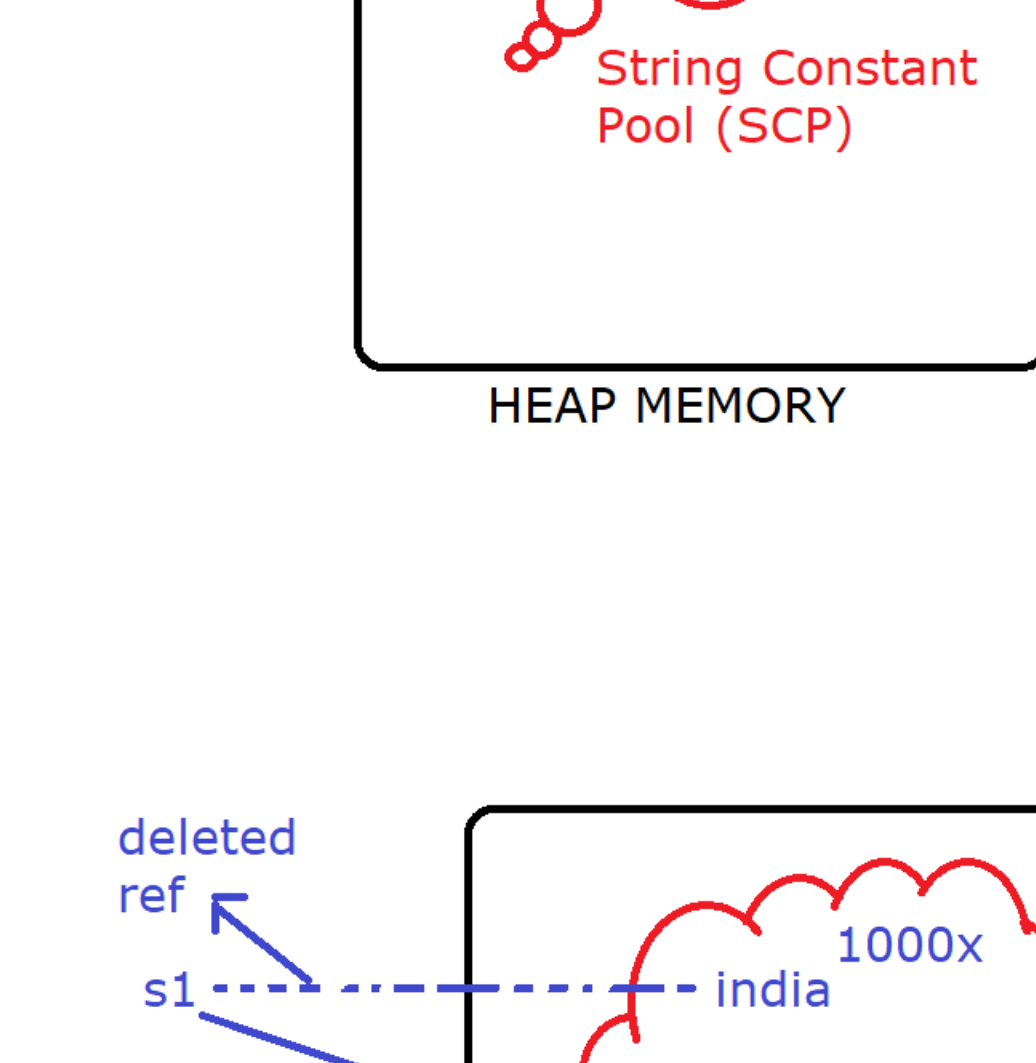
In java String can be created by using 3 ways :-

- 1) By using String Literal  
String x = "Ravi";
- 2) By using new keyword  
String y = new String("Hyderabad");
- 3) By using character array  
char z[] = {'H','E','L','L','O'};

## Immutability in String :

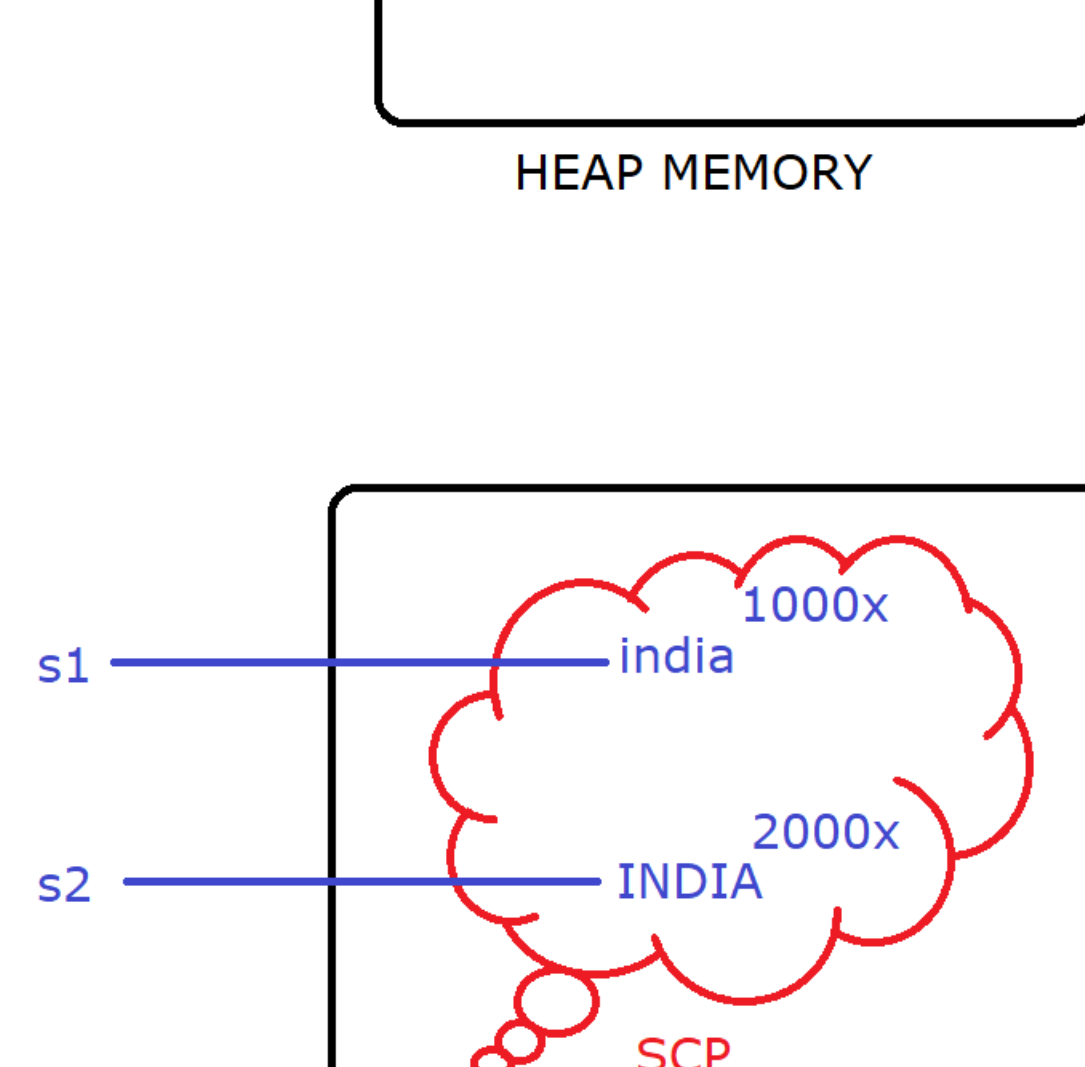
Case 1 :

```
String s1 = "india";  
s1.toUpperCase();  
IO.println(s1); //india
```



Case 2 :

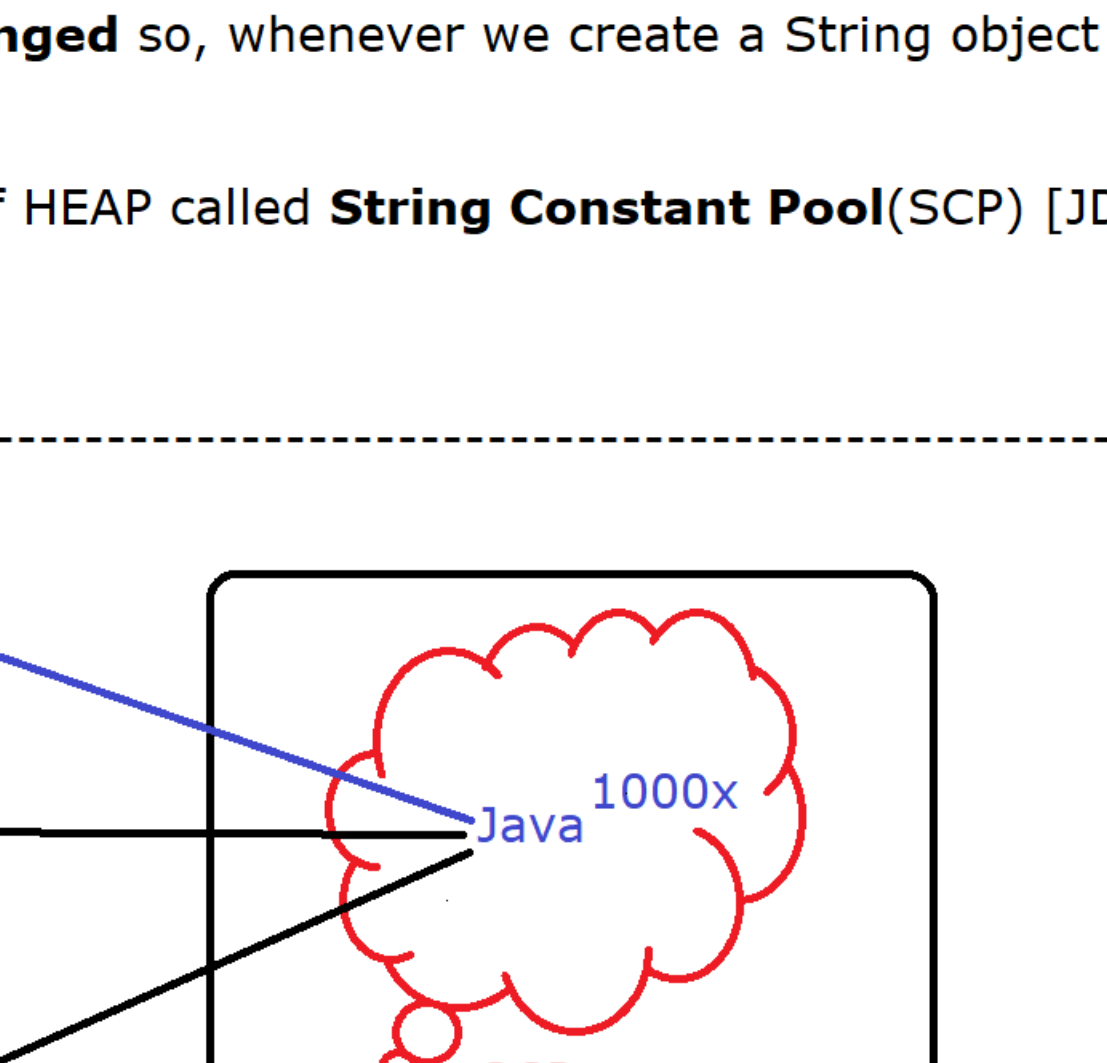
```
String s1 = "india";  
s1 = s1.toUpperCase();  
IO.println(s1);
```



Case 3:

```
String s1 = "india";  
String s2 = s1.toUpperCase();  
IO.println(s1); //india  
IO.println(s2); //INDIA
```

Note : s1 -> reference variable  
india -> String Object  
"" -> Intern()



In java Strings Objects are **immutable means unchanged** so, whenever we create a String object in java it can't be modifiable.

Strings literals are created in a very special memory of HEAP called **String Constant Pool(SCP)** [JDK 1.8 Metaspace] and it is not eligible for garbage collection.

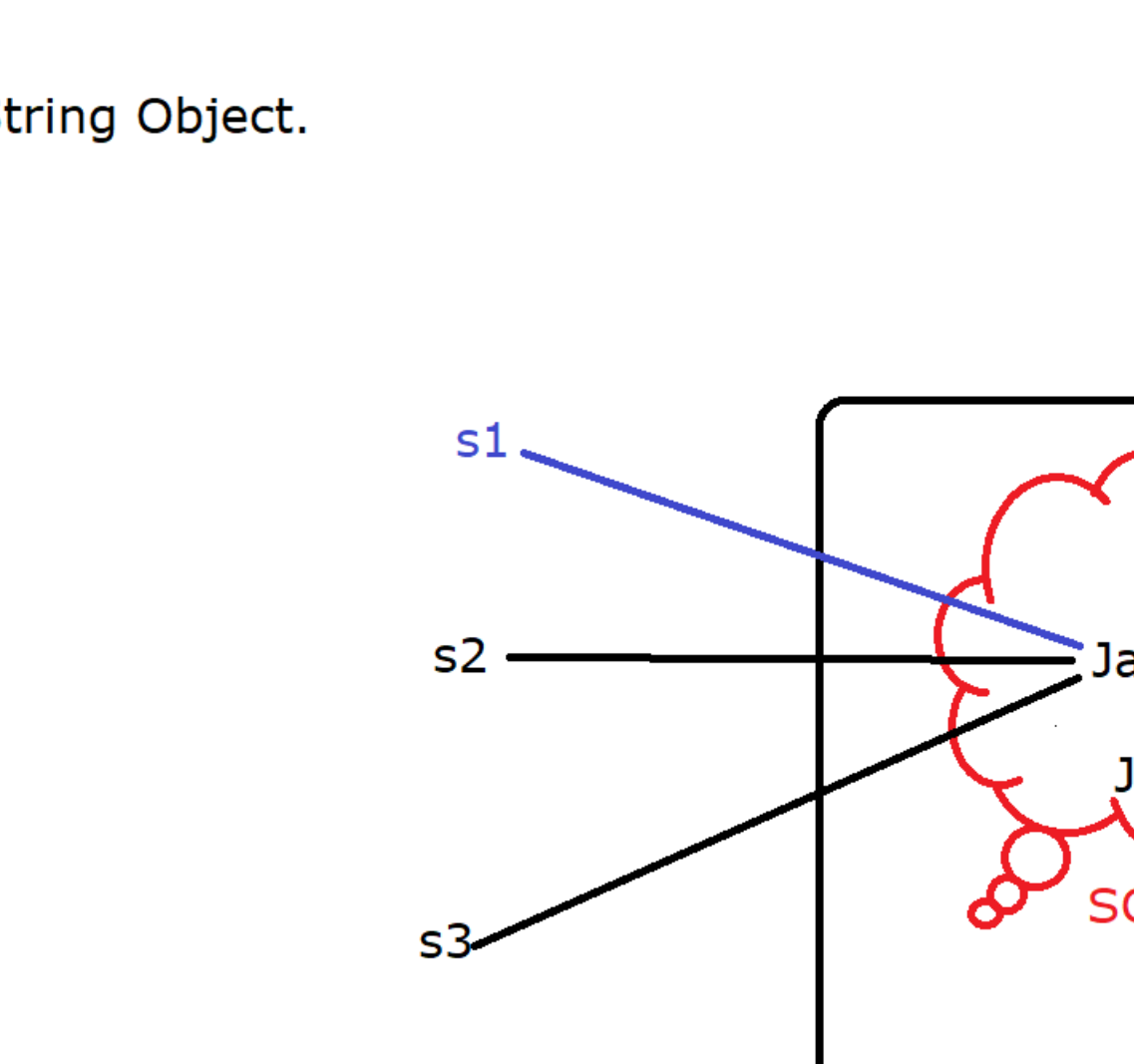
String once created can't be modifiable.

## Facts about String and memory

String s1 = "Java";

String s2 = "Java";

String s3 = "Java";



## Facts about String and memory :-

In java Whenever we create a new String object by using String literal, first of all JVM will verify whether the String we want to create is pre-existing (already available ) in the String constant pool or not.

If the String is pre-existing (already available) in the String Constant pool then JVM WILL NOT CREATE ANY NEW STRING OBJECT, the same old existing String object would be refer by new reference variable as shown in the diagram above diagram.

Note :- In SCP area we can't have duplicate String Object.

## \*\*\*Why Strings are immutable :

String s1 = "Java";

String s2 = "Java";

String s3 = "Java";  
s3.toUpperCase();



- 1) As we know a String object in the String constant pool can be refer by multiple reference variables, if any of the reference variable will modify the String Object value then it would be very difficult for the another reference variables pointing to same String object to get the original value, what they have defined earlier as shown in the diagram.
- 2) While working with Collections Framework, String & Wrapper classes we are using as a HashMap key so It is strongly recommended that HashMap key must be immutable, It is also a reason why String & Wrapper classes are immutable.
- 3) Strings are immutable so it provides SECURITY & THREAD SAFETY.

Strings are widely used for sensitive values such as Fully Qualified class names, file paths, URLs, database connection URL and so on.

Immutability prevents another part of the program from changing the original value after it has been validated or shared. It also useful when two threads want to share String data but no one can modify due to immutability.